

Universidade Estadual de Mato Grosso do Sul

Ciência da Computação

Algoritmos e Estruturas de Dados I (AED-I)

Array Multidimensional

Prof. Nilton

nilton@comp.uems.br

Array Multidimensional

- **Pode-se criar um array de array;**
- **Mais comum é o bidimensional;**
- **Por exemplo: `int matriz[4][3]`, tem-se um array bidimensional de 4 linhas e 3 colunas;**
- **Todos os elementos do array são do mesmo tipo e por isto a estrutura é dita homogênea;**
- **Deve-se definir a quantidade de linhas e colunas antes de usar um array.**

Array Multidimensional

- Em C as **linhas** do array são numeradas a partir do ZERO “0, 1, 2, ..., L - 1”, onde **L** é a quantidade de linhas;
- Em C as **colunas** do array são numeradas a partir do ZERO “0, 1, 2, ..., C - 1”, onde **C** é a quantidade de colunas;
- Para o array “*int matriz[4][3]*”

	colunas $\Rightarrow$ 0	1	2
0	matriz[0][0]	matriz[0][1]	matriz[0][2]
1	matriz[1][0]	matriz[1][1]	matriz[1][2]
2	matriz[2][0]	matriz[2][1]	matriz[2][2]
3	matriz[3][0]	matriz[3][1]	matriz[3][2]

↑
linhas

Array Bidimensional

- **Declarando, inicializando e alterando**

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {  
    int matriz[3][2];          /* Declara uma matriz 3x2 */  
    int i, j;  
  
    for (i = 0; i < 3; i++) {  
        for (j = 0; j < 2; j++) {  
            matriz[i][j] = 0;  /* inicializando com 0 a matriz */  
        }  
    }  
  
    matriz[0][0] = 1; /* Atribui o valor 1 ao canto superior esquerdo */  
    matriz[2][1] = 7; /* Atribui o valor 7 ao canto inferior direito */  
  
    return 0;  
}
```

Matriz	
1	0
0	0
0	7

Array Bidimensional - usando sizeof

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {  
    int matriz[3][2];      /* Declara uma matriz 3x2 */  
    int i, j;  
  
    for (i = 0; i < sizeof(matriz) / sizeof(matriz[i]); i++) {  
        for (j = 0; j < sizeof(matriz[i]) / sizeof(int); j++) {  
            matriz[i][j] = 0; /* inicializando */  
        }  
    }  
  
    matriz[0][0] = 1; /* Atribui o valor 1 ao canto superior esquerdo */  
    matriz[2][1] = 7; /* Atribui o valor 7 ao canto inferior direito */  
  
    return 0;  
}
```

Array Bidimensional - lendo e escrevendo

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {  
    int matriz[3][2];    /* Declara uma matriz 3x2 */  
    int i, j;  
  
    for (i = 0; i < sizeof(matriz)/sizeof(matriz[i]); i++) {  
        for (j = 0; j < sizeof(matriz[i])/sizeof(int); j++) {  
            printf("Digite matriz[%d][%d]: ", i, j);  
            scanf("%d",&matriz[i][j]);    /* Lendo */  
        }  
    }  
  
    printf("\n Matriz\n");  
    for (i = 0; i < sizeof(matriz)/sizeof(matriz[i]); i++) {  
        for (j = 0; j < sizeof(matriz[i])/sizeof(int); j++) {  
            printf(" %d\t", matriz[i][j]);    /* Escrevendo */  
        }  
        printf("\n");  
    }  
    return 0;  
}
```

```
Digite matriz[0][0]: 1  
Digite matriz[0][1]: 2  
Digite matriz[1][0]: 3  
Digite matriz[1][1]: 4  
Digite matriz[2][0]: 5  
Digite matriz[2][1]: 6
```

Matriz

1	2
3	4
5	6

Copiando um Array Bidimensional

- **Observe declarando e inicializando**

```
int mA[2][5] = { {1,2,3,4,5}, {6,7,8,9,10} };  
int mB[2][5];
```

- **Cópia **incorreta****

```
mB = mA;  
mB[x] = mA[x];
```

- **Cópia **correta****

```
for (i = 0; i < 2; i++) {  
    for (j = 0; j < 5; j++) {  
        mB[i][j] = mA[i][j];  
    }  
}
```

Matriz A (2x5)				
1	2	3	4	5
6	7	8	9	10
Matriz B (2x5)				
1	2	3	4	5
6	7	8	9	10

Strings em um Array Bidimensional

- **Declarando – um array de strings**

```
#define NUM_DE_STRINGS 5 //dá um nome para a constante 5
#define COMPR_DAS_STRINGS 50
char strings[NUM_DE_STRINGS][COMPR_DAS_STRINGS]; //declaração
```

- **Lendo**

```
for (count=0;count<NUM_DE_STRINGS;count++) {
    printf ("\n\nDigite uma string: ");
    gets (strings[count]); //lê uma palavra e guarda em uma linha do array bidimensional
}
```

- **Escrevendo**

```
printf ("\n\n\nAs strings que voce digitou foram:\n\n");
for (count=0;count<NUM_DE_STRINGS;count++)
    printf ("%s\n",strings[count]);
```

```
Digite uma string: Primeira String
Digite uma string: Segunda String
Digite uma string: Terceira
Digite uma string: e outra String
Digite uma string: ultima String

As strings que voce digitou foram:

Primeira String
Segunda String
Terceira
e outra String
ultima String
```


Array Bidimensional - Declarando e inicializando Strings

- **Ex1**

```
char universidades[5][10]={"UEMS","PUC-PR","UFMS","PUC-MG","UFGD"};
```

Array bidimensional com a capacidade de 5 strings e cada string de tamanho máximo de 10 caracteres. Lembre-se que no final de cada string tem-se o ‘\0’, que indica onde termina a string, e ele ocupa 1 (uma) posição da string.

- **Ex2**

```
char sinal[][15] = {"vermelho","laranja","verde"};
```

Quando o programador não informa a capacidade da 1ª dimensão do array bidimensional o compilador quem insere essa capacidade e neste caso é 3, que cabem no máximo 3 strings.

- **Ex3**

```
char logico[][11] = { {"verdadeiro"},  
                      {"falso"} };
```

- **Ex4**

```
char boolean[][6] = { {'t','r','u','e','\0'},  
                      {'f','a','l','s','e','\0'} };
```

A inicialização de cada string é caracter a caracter e neste caso o programador quem deve colocar o finalizador da string (‘\0’). Observe que a string com a maior quantidade de caracteres é “false”, 5 caracteres da palavra + 1 finalizador igual a 6, que coincide com a capacidade máxima da 2ª dimensão do array bidimensional.

Array Tridimensional

```
#include <stdio.h>
```

```
#define xx 3
```

```
#define yy 3
```

```
#define zz 3
```

```
int main(int argc, char *argv[]) {
```

```
    int magic_cube[xx][yy][zz];
```

```
    int x,y,z, n=0;
```

```
    for (x=0;x<xx;x++)
```

```
        for (y=0;y<yy;y++)
```

```
            for (z=0;z<zz;z++)
```

```
                magic_cube[x][y][z] = ++n;
```

```
    for (x=0;x<xx;x++)
```

```
        for (y=0;y<yy;y++)
```

```
            for (z=0;z<zz;z++)
```

```
                printf("\n[%d][%d][%d]=%d ",x,y,z,magic_cube[x][y][z]);
```

```
    printf("\n\n");
```

```
    return 0;
```

```
}
```

